

Porting Plan: U(1)-Symmetric Tensor Networks for Quench_dyn

File-by-file specification for charge-conserving DMRG and TDVP
on the 1D XXZ chain

Nishan Ranabhat

June 15, 2026

Abstract

This document specifies, in detail, the porting of the U(1)-symmetric tensor-network machinery already implemented in `KagomeDSF/U1/` to the 1D XXZ project `Quench_dyn`. The motivation is the quench-dynamics study laid out in `docs/manuscript/XXZ_phase_ordering.tex`: reaching the regime where the even-odd splitting $\Delta_{\text{eo}}(N) \sim e^{-N/\xi}$ becomes smaller than the random-field-induced mixing requires larger N and larger Δ than dense ED or dense MPS can access. Exploiting total- S^z conservation gives a 3–10× reduction in memory and runtime at fixed χ , equivalently a larger effective χ at fixed cost. No new algorithms are needed; the existing U(1) layer is generic and the only project-specific code is a ~ 300 LOC XXZ MPO builder. The plan below identifies (i) which files are copied verbatim, (ii) what new native code is written, (iii) the test plan against the dense baseline and ED, and (iv) the phased order of work. Implementation is deferred to a separate working directory.

Contents

1	Motivation and scope	1
2	Symmetry analysis specific to our Hamiltonian	2
2.1	Conservation of total S^z	2
2.2	Charge convention	2
2.3	Bond charges on the XXZ MPO	2
2.4	The $m = 0$ Néel branches and what U(1) does <i>not</i> resolve	3
3	Inventory of the existing U(1) layer	3
3.1	Key types and conventions to inherit	4
3.2	Operations already implemented	4
4	Target architecture for the port	5
4.1	Where the new code lives	5
4.2	Why a sub-package and not a flat add	5
4.3	Renaming considerations during the copy	5
5	Phase A–D: portable layers (copy verbatim)	6
5.1	Phase A — Symmetry (3 files)	6
5.2	Phase B — Tensor (5 files)	6

5.3	Phase C — MPS operations (4 files)	6
5.4	Phase D — Algorithms (2 files)	7
6	Phase E — Native: the XXZ charged MPO	9
6.1	File: U1/src/Models/xxz.jl	9
7	Phase F — Native: Néel state and quench-protocol initialisers	10
8	Module integration and exports	11
9	Validation plan	12
9.1	Inherited generic tests (copy + rename)	13
9.2	New XXZ-specific tests	13
9.3	Drift benchmarks	14
10	Open questions and risks	14
10.1	MPO convention drift	14
10.2	Padding compatibility for 1-site TDVP	14
10.3	Numerical type uniformity	14
10.4	Performance expectations	15
10.5	Bunching the $q = 0$ bond states	15
11	Phased schedule	16
12	What this plan does <i>not</i> commit to	16
13	Summary	17

1 Motivation and scope

The companion manuscript shows that the late-time Néel order following the quench

$$H_i = H_{\text{XXZ}}(\Delta_i) - \sum_j h_j S_j^z \longrightarrow H_f = H_{\text{XXZ}}(\Delta_f)$$

is parity-controlled: the local magnetization $\langle S_i^z(t) \rangle$ is purely a cross-parity matrix element and scales as $\mathcal{O}(h)$ as long as the random-field mixing $|\langle \text{even} | V | \text{odd} \rangle|$ is small compared with the even-odd splitting $\Delta_{\text{eo}}(N) = 2|t|$. In the deep ordered regime the tunneling between the two Néel branches behaves as $t \sim J(1/\Delta)^{cN} \sim e^{-N/\xi}$. Reaching the crossover $|v| \gtrsim \Delta_{\text{eo}}(N)$ where spontaneous symmetry breaking should become visible therefore requires *both* large Δ and large N . ED is restricted to $N \lesssim 20$; dense MPS is bottlenecked by the bond dimension χ required to resolve the long-range entanglement built up by the post-quench dynamics.

Both H_i and H_f commute with the total magnetization $S_{\text{tot}}^z = \sum_j S_j^z$, including the disorder term $V = -\sum_j h_j S_j^z$, which is diagonal in S^z . The entire protocol therefore stays inside the $m = 0$ sector (for even N), exactly the sector that contains the two Néel branches $|A\rangle = |\uparrow\downarrow\dots\rangle$ and $|B\rangle = |\downarrow\uparrow\dots\rangle$. Restricting MPS/MPO tensors to the block-sparse U(1) representation is a strict gain with no physics lost. The remaining $\mathbb{Z}_2$ parity inside the $m = 0$ sector is left intact as an open symmetry one can use as a diagnostic at the analysis stage.

Why a port rather than a rewrite. The U(1) layer already exists at `/home/nishan/UVA_postdoc/Research/K`. It was developed for the sawtooth/Kagome strip, which is genuinely harder than 1D XXZ (super-sites of dimension 4, MPO bond dimension up to $\chi_{\text{MPO}} = 5$ with mixed charge content, two-sublattice geometry). Of ~ 6600 lines of Julia, roughly 5400 are generic block-sparse machinery and only ~ 500 are sawtooth-specific. The XXZ chain is a strict subset of that geometry: spin- $\frac{1}{2}$ physical sites and a $\chi_{\text{MPO}} = 5$ bond with the same five FSM states (Id-start, S^+ -pending, S^- -pending, S^z -pending, Id-finish) carrying charges $(0, +2, -2, 0, 0)$. The dense XXZ MPO builder already in `Quench_dyn` (`src/Builders/xxz_builder.jl`) uses exactly these bond states with the upper-triangular convention, so the U(1) projection is mechanical.

Out of scope. This document plans the U(1) port only. We do *not* remove or modify the existing dense code under `src/`; the dense path remains the reference implementation for validation and the small- N workflow. We do *not* change algorithm choices — the existing DMRG, two-site TDVP, and one-site TDVP carry over with the same APIs. We do *not* add new symmetries (parity, $\mathbb{Z}_n$) at this stage; the U(1) layer is already written generically over an `AbstractSymmetry` type and can be extended later.

2 Symmetry analysis specific to our Hamiltonian

2.1 Conservation of total S^z

The XXZ Hamiltonian

$$H_{\text{XXZ}} = J \sum_i (S_i^x S_{i+1}^x + S_i^y S_{i+1}^y) + \Delta \sum_i S_i^z S_{i+1}^z$$

has XY part $\frac{J}{2} \sum_i (S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+)$ which flips one spin up and a neighbouring spin down — total S^z unchanged. The Ising part is diagonal in S^z , hence commutes trivially. The disorder term $V = -\sum_j h_j S_j^z$ is also diagonal in S^z . Therefore

$$[H_i, S_{\text{tot}}^z] = [H_f, S_{\text{tot}}^z] = 0,$$

and the time evolution remains in a fixed magnetization sector throughout both the imaginary-time preparation (DMRG) and the real-time quench (TDVP).

2.2 Charge convention

Throughout we use the integer charge

$$q = 2S^z \in \mathbb{Z}.$$

For spin- $\frac{1}{2}$, $|\uparrow\rangle \leftrightarrow q = +1$ and $|\downarrow\rangle \leftrightarrow q = -1$. The action of S^+ raises q by $+2$; S^- lowers by -2 ; S^z preserves q . This is the TeNPy convention adopted by the existing U(1) layer (`KagomeDSF/U1/src/Symmetry/chargeinfo.jl`).

2.3 Bond charges on the XXZ MPO

The dense XXZ MPO in `Quench_dyn` uses a $\chi_{\text{MPO}} = 5$ upper-triangular bond with the FSM states already documented inline at `src/Builders/xxz_builder.jl:47--63`:

qindex	FSM state	bond charge q
1	Id-start (no coupling in flight)	0
2	S^+ -placed (awaiting S^- to the right)	+2
3	S^- -placed (awaiting S^+ to the right)	-2
4	S^z -placed (awaiting S^z to the right)	0
5	Id-finish (coupling completed)	0

With the U(1) layer’s per-tensor rule on the MPO $W[w_L, w_R, p, p^*]$,

$$\zeta_{w_L} q_{w_L} + \zeta_{w_R} q_{w_R} + \zeta_p q_p + \zeta_{p^*} q_{p^*} = 0, \quad \zeta = (+1, -1, +1, -1),$$

each non-zero entry of the dense W already constructed by `build_xxz_mpo` satisfies the rule. Two spot checks:

- $W[1, 2, \uparrow, \downarrow] = (J/2) S^+$: $0 - 2 + 1 - (-1) = 0 \checkmark$
- $W[2, 5, \downarrow, \uparrow] = S^-$: $+2 - 0 + (-1) - 1 = 0 \checkmark$

So the U(1) projection of the existing dense MPO is well-defined and the “dense-then-from_dense” construction used in `KagomeDSF/U1/src/Models/sawtooth.jl` carries over verbatim.

2.4 The $m = 0$ Néel branches and what U(1) does *not* resolve

Both Néel branches $|A\rangle$ and $|B\rangle$ have $S_{\text{tot}}^z = 0$ for even N ; they live in the same U(1) block. Parity $P : S_j^z \mapsto -S_j^z$ maps $A \leftrightarrow B$ *within* the $m = 0$ sector but maps $m \leftrightarrow -m$ between sectors — so parity does not commute with S_{tot}^z globally. Inside $m = 0$ alone, parity acts as a residual $\mathbb{Z}_2$. We do not enforce it as a symmetry in the tensor structure; the even/odd splitting is what we are trying to *resolve*, not symmetrize away.

3 Inventory of the existing U(1) layer

The U(1) package at `KagomeDSF/U1/` is self-contained: `stdlib` only (`LinearAlgebra`, `Random`, `Printf`, `TOML`) plus `JLD2` for serialization. It declares `TensorOperations` in `Project.toml` but does not actually import it inside the U(1) code. The top-level layout (cf. `KagomeDSF/U1/src/U1.jl`):

Directory	LOC	Purpose
<code>Symmetry/</code>	~ 900	Charge metadata, leg-charge bookkeeping, leg pipes (fusion).
<code>Tensor/</code>	~ 2000	<code>ChargedTensor</code> type, dense/charged adapters, <code>linalg</code> , contractions, SVD/QR.
<code>MPS/</code>	~ 1050	Environments, canonicalization, generic builders, padding, IO.
<code>Algorithms/</code>	~ 1000	Effective Hamiltonians, Lanczos, Krylov-exponential, DMRG, TDVP (1- and 2-site).
<code>Models/</code>	~ 500	Sawtooth-specific MPO and site builders. <i>Not portable</i> .
<code>DSF/</code>	~ 650	Spectral-function pipeline. <i>Not needed for quench dynamics</i> .
<code>ED/</code>	~ 600	Dense ED for sawtooth, validation only. <i>Not needed</i> .
<code>Parallel/</code>	–	Threaded reimplementations of hot paths. <i>Optional, post-MVP</i> .

3.1 Key types and conventions to inherit

Charge metadata (Symmetry/chargeinfo.jl).

```
abstract type AbstractSymmetry end
struct U1Symmetry <: AbstractSymmetry end

struct ChargeInfo{S<:AbstractSymmetry}
    mod::Int      # 0 for U(1)
    name::Symbol  # :twice_Sz
end
```

Leg charges (Symmetry/legcharge.jl).

```
struct LegCharge{S<:AbstractSymmetry}
    chinfo::ChargeInfo{S}
    charges::Vector{Int}      # one per qindex (block)
    slices::Vector{Int}       # block boundaries, length nblocks+1
    qconj::Int8               # +-1
end
```

Block-sparse tensor (Tensor/chargedtensor.jl).

```
struct ChargedTensor{T, N, S<:AbstractSymmetry}
    legs::NTuple{N, LegCharge{S}}
    qtotal::Int
    qdata::Matrix{Int}        # (nblocks_stored, N) of qindices
    data::Vector{Array{T, N}} # the block contents
end
```

Sign convention (the *single* thing that matters most when porting):

Tensor	Legs	ζ
MPS site	(v_L, p, v_R)	$(+1, +1, -1)$
MPO site	(w_L, w_R, p, p^*)	$(+1, -1, +1, -1)$
Left env	$(\bar{v}, w, v)$	$(+1, -1, -1)$
Right env	$(\bar{v}, w, v)$	$(-1, +1, +1)$

with the per-tensor charge rule $\sum_k \zeta_k q_{k, \text{qid}\mathbf{x}[k]} \equiv Q \pmod{\text{mod}}$ for the stored qindex tuples.

3.2 Operations already implemented

- **Adapters.** `from_dense(A, legs, qtotal; tol)` projects a dense array onto its allowed-sector blocks, warning if discarded mass exceeds `tol`. `to_dense(t)` inflates back.
- **Linalg.** `conj`, `norm`, `dot`, scalar ops, `axpby`; everything per-block.
- **Contractions.** `permute`, block-sparse `tensordot`, `combine_legs/split_legs` via `LegPipe`.
- **Decompositions.** `svd_charged` with per-block σ -floor, global sort, relative cutoff, hard cap on χ . `qr_charged` for canonicalization.
- **MPS ops.** `trivial_left_env`, `update_left_env`, symmetric right-side variants, `build_environments`, `make_canonical!`, orthogonality checks, `measure_norm`, `measure_energy`, `pad_mps_charged` (charge-preserving).

- **Solvers.** `TwoSiteEffectiveH`, `OneSiteEffectiveH`, `ZeroSiteEffectiveH`, each with `_apply`; `LanczosSolver`, `KrylovExponential`, each acting directly on `ChargedTensor`.
- **Algorithms.** `ChargedMPSSState` container, `dmrg_sweep!`, `tdvp_sweep!` (2-site), `tdvp_sweep_one_site!` (1-site).

4 Target architecture for the port

4.1 Where the new code lives

We propose creating a sibling sub-package under `Quench_dyn/` that mirrors the `KagomeDSF/U1/` layout:

```
Quench_dyn/
  src/                <-- existing dense code, untouched
  U1/                 <-- NEW sub-package, self-contained
    Project.toml
    src/
      QuenchDynU1.jl  <-- module top, includes everything below
      Symmetry/       <-- copy from KagomeDSF/U1/src/Symmetry/
      Tensor/         <-- copy from KagomeDSF/U1/src/Tensor/
      MPS/            <-- copy from KagomeDSF/U1/src/MPS/
      Algorithms/     <-- copy from KagomeDSF/U1/src/Algorithms/
      Models/
        xxz.jl        <-- NEW (~300 LOC), replaces sawtooth.jl
    test/
      ... (copied generic tests + new XXZ tests)
```

This keeps the dense and U(1) code paths fully decoupled. The existing scripts in `scripts/` continue to work against `QuenchDyn`; new scripts that want U(1) load `QuenchDynU1` instead. Once the U(1) path is stable we can decide whether to merge them under one umbrella module; that is deferred.

4.2 Why a sub-package and not a flat add

Two reasons. First, it mirrors how `KagomeDSF/U1/` was structured — proven to work, so we inherit the dependency hygiene for free. Second, a sub-package has its own `Project.toml` and tests, so the U(1) layer can be precompiled and tested independently of the dense code. This matches your modular-architecture preference (no black-box pipelines, every operation independent).

4.3 Renaming considerations during the copy

None. Every type, function, and convention in the U(1) layer is XXZ-agnostic already. The module name (U1 vs `QuenchDynU1`) is the only thing that needs to change at the top of `src/QuenchDynU1.jl`, and that is just an edit to the `module` keyword and the file header.

5 Phase A–D: portable layers (copy verbatim)

5.1 Phase A — Symmetry (3 files)

File	LOC	Notes
Symmetry/chargeinfo.jl	159	AbstractSymmetry, NoSymmetry, U1Symmetry, ChargeInfo, make_valid. Copy as-is.
Symmetry/legcharge.jl	~ 340	LegCharge, from_qflat, from_trivial, block_size(s), slice, charge, findblock, is_contractible. Copy as-is.
Symmetry/legpipe.jl	~ 400	LegPipe, q_map, strides, helpers used by combine_legs. Copy as-is.

5.2 Phase B — Tensor (5 files)

File	LOC	Notes
Tensor/chargedtensor.jl	~ 450	The ChargedTensor{T,N,S} type and unchecked/checked constructors.
Tensor/adapters.jl	~ 250	validate_charges, from_dense, to_dense.
Tensor/linalg.jl	~ 200	conj, copy, norm, dot, scalar ops, axpby.
Tensor/contraction.jl	~ 600	permute, tensordot, combine_legs, split_legs.
Tensor/decomposition.jl	~ 500	svd_charged, qr_charged, truncation policy.

5.3 Phase C — MPS operations (4 files)

File	LOC	Notes
MPS/tensorops.jl	~ 450	trivial_left_env/right_env, update_left_env, symmetric right, build_environments, move_orthogonality*!, make_canonical!, is_left/right_orthogonal, measure_norm, measure_energy.
MPS/builders.jl	~ 350	Generic spin_half_leg, trivial_boundary_leg, product_state_charged, random_charged_mps, rand_charged_tensor. Sawtooth-specific functions (sawtooth_supersite_leg, sawtooth_mpo_bond_leg, sawtooth_random_charged_mps, sawtooth_supersite_operator) should be deleted on copy — they belong in Models/.
MPS/io.jl	~ 150	save_charged_mps, load_charged_mps. Copy as-is.
MPS/padding.jl	~ 100	pad_mps_charged. Charge-preserving padding; essential for the 1-site TDVP workflow.

One pruning during copy. MPS/builders.jl mixes generic constructors with sawtooth super-site helpers. The split is clean: anything named `sawtooth_*` goes; anything else is generic and stays. The generic `product_state_charged` takes a physical `LegCharge` and a vector of basis indices — exactly what we need to construct Néel states for the XXZ chain (Section 7).

5.4 Phase D — Algorithms (2 files)

File	LOC	Notes
Algorithms/solvers.jl	~ 450	EffectiveHamiltonian hierarchy (zero-/one-/two-site), _apply, LanczosSolver (_solve), KrylovExponential (_evolve).
Algorithms/algorithms.jl	~ 550	ChargedMPSState, dmrg_sweep!, tdvp_sweep! (2-site), tdvp_sweep_one_site!.

API mapping ($\text{dense} \rightarrow \text{U}(1)$).

Dense (QuenchDyn)	U(1) (QuenchDynU1)	Differences
MPS{T}	Vector{ChargedTensor{T,3,U1Symmetry}}	No wrapper struct in U(1); a bare Vector.
MPO{T}	Vector{ChargedTensor{T,4,U1Symmetry}}	Same.
Environment{T}	Vector{Union{Nothing, ChargedTensor{T,3,U1Symmetry}}}	Same.
MPSState	ChargedMPSState{Tmps,Tmpo,Tenv,S}	Carries explicit element types; same role.
DMRGOptions	passed as (chi_max, cutoff) kwargs	U(1) inline the options into dmrg_sweep!.
TDVPOptions	passed as (dt, chi_max, cutoff) kwargs	Same comment.
dmrg_sweep	dmrg_sweep!	In-place in U(1) (matches existing convention there).
tdvp_sweep	tdvp_sweep!	In-place; otherwise identical sweep mechanics.
tdvp_sweep_one_site	tdvp_sweep_one_site!	Same

The `!` naming difference (mutating in `U(1)` vs returning new in `dense`) is a real semantic distinction; we keep `U(1)`'s convention as it is and *do not* rewrite the `dense` routines.

6 Phase E — Native: the XXZ charged MPO

This is the only piece of model-specific code we write. The strategy mirrors `KagomeDSF/U1/src/Models/sawtooth.` build the dense W tensors with the existing routine, declare the leg-charge structure, and project with `from_dense`.

6.1 File: `U1/src/Models/xxz.jl`

Public functions.

```
"""
    build_xxz_mpo_charged(N, J, Delta, h::Vector{Float64};
                          T::Type = Float64) -> Vector{ChargedTensor{T,4,
                          U1Symmetry}}

U(1)-charge-conserving XXZ MPO with chi_MPO = 5 and per-site longitudinal
field h_i. Charge convention: q = 2 S^z. Bond states and charges:
    [Id-start, S+-pending, S--pending, Sz-pending, Id-finish]
    charges = [0, +2, -2, 0, 0]
"""
function build_xxz_mpo_charged(N::Int, J::Float64, Delta::Float64,
                               h::Vector{Float64};
                               T::Type = Float64)

end

# Convenience overloads mirroring the dense API:
build_xxz_mpo_charged(N, J, Delta, h_uniform::Float64; T=Float64)
build_xxz_mpo_charged(N, J, Delta; T=Float64)
```

Leg constructors.

```
"""Physical spin-1/2 leg with charges (+1, -1). Bunched (no degeneracy)."""
xxz_phys_leg(ci::ChargeInfo{U1Symmetry}; qconj=+1) =
    LegCharge(ci, [(+1, 1), (-1, 1)], qconj)

"""chi_MPO = 5 XXZ bond leg in unbunched form: five qindices, each dim 1,
charges (0, +2, -2, 0, 0) corresponding to the FSM states above."""
xxz_mpo_bond_leg(ci::ChargeInfo{U1Symmetry}; qconj=+1) =
    LegCharge(ci, [0, +2, -2, 0, 0], [1, 2, 3, 4, 5, 6], qconj)
```

The bunched physical leg `xxz_phys_leg` is essentially an alias of the existing generic `spin_half_leg` from `MPS/builders.jl`. Defining it locally inside `Models/xxz.jl` keeps the XXZ vocabulary self-contained and avoids reaching into the generic namespace from physics code. Bunching the three $q=0$ bond states (states 1, 4, 5) is a $\sim 1.5\times$ speedup on W -block contractions; the sawtooth blueprint explicitly leaves this as a post-MVP refinement, so we follow the same policy and keep the five states unbunched in the initial port.

MPO assembly (pseudocode).

```
function build_xxz_mpo_charged(N, J, Delta, h::Vector{Float64};
                               T::Type = Float64)

    @assert length(h) == N
    @assert N >= 2
```

```

# 1. Dense reference: reuse Quench_dyn's existing builder.
W_dense = QuenchDyn.build_xxz_mpo(N, J, Delta, h; T = T).tensors
# W_dense is a Vector of 4-arrays sized (chi_l, chi_r, d, d), with
# boundary tensors already sliced to (1, 5, d, d) and (5, 1, d, d).

# 2. Leg structure.
ci      = ChargeInfo{U1Symmetry}(0, :twice_Sz)
phys    = xxz_phys_leg(ci)
bond_boundary = trivial_boundary_leg(ci)    # dim-1, q = 0
bond_bulk = xxz_mpo_bond_leg(ci)

legs_first = (bond_boundary, conj(bond_bulk), phys, conj(phys))
legs_bulk  = (bond_bulk,      conj(bond_bulk), phys, conj(phys))
legs_last  = (bond_bulk,      conj(bond_boundary), phys, conj(phys))

# 3. Project to U(1) blocks. Every non-zero entry of the dense W
# satisfies the charge rule, so discarded mass should be ~0
# (warn threshold = 1e-12).
mpo = Vector{ChargedTensor{T, 4, U1Symmetry}}(undef, N)
mpo[1] = from_dense(W_dense[1], legs_first, 0)
for i in 2:(N - 1)
    mpo[i] = from_dense(W_dense[i], legs_bulk, 0)
end
mpo[N] = from_dense(W_dense[N], legs_last, 0)
return mpo
end

```

Per-site vs shared tensors. Site-dependent h_i means we cannot share the same bulk tensor across sites (as the sawtooth builder does); each site needs its own `from_dense` call. This is fine — the dense XXZ builder already constructs N distinct tensors. For the special case $h_i \equiv h_0$ (uniform field) one could share a single bulk `ChargedTensor` reference, but the memory saving is negligible ($\chi_{\text{MPO}} = 5$, $d = 2$ means each W is at most $5 \cdot 5 \cdot 4 = 100$ floats). We do not optimize this.

Cross-dependency note. The pseudocode above shows `QuenchDynU1.build_xxz_mpo_charged` calling `QuenchDyn.build_xxz_mpo` to reuse the dense W tensors. This introduces a one-way dependency `QuenchDynU1 → QuenchDyn`. If we want `QuenchDynU1` to be standalone (mirroring how `KagomeDSF/U1` is standalone) we should instead inline the dense W construction — ~ 50 LOC — into `Models/xxz.jl`. Recommendation: inline. It removes the cross-dep, the code is small, and the two builders can diverge in future without coupling.

7 Phase F — Native: Néel state and quench-protocol initialisers

Néel product state in the $m = 0$ sector.

```

"""
    xxz_neel_state(::Type{T}, N::Int; start_up::Bool = true)
    -> Vector{ChargedTensor{T, 3, U1Symmetry}}

The charge-conserving Neel product state in the m = 0 sector of the
spin-1/2 chain. start_up = true gives |up dn up dn ...>, false gives
|dn up dn up ...>. Requires even N.
"""
function xxz_neel_state(::Type{T}, N::Int; start_up::Bool = true) where T
    iseven(N) || throw(ArgumentError(
        "xxz_neel_state needs even N for m = 0; got N = $N."))

```

```

ci    = ChargeInfo{U1Symmetry}(0, :twice_Sz)
phys = xxz_phys_leg(ci)
# phys basis indices: 1 = up (q=+1), 2 = dn (q=-1)
indices = start_up ? [iseven(i) ? 2 : 1 for i in 1:N] :
                  [iseven(i) ? 1 : 2 for i in 1:N]
return product_state_charged(T, phys, indices)
end

```

The two Néel branches $|A\rangle$ and $|B\rangle$ in the manuscript correspond to `start_up = true` and `false` respectively.

Random charged MPS in a target sector. We expose a thin wrapper around the existing generic constructor:

```

"""
    xxz_random_charged_mps(::Type{T}, N::Int, chi::Int;
                          qtotal::Int = 0) -> Vector{ChargedTensor{T,3,
                          U1Symmetry}}

Random charge-conserving spin-1/2 MPS with total charge qtotal (q = 2*S^z_tot).
Used for initial DMRG warm starts and as a richer-than-product seed for
benchmarking convergence.
"""
function xxz_random_charged_mps(::Type{T}, N::Int, chi::Int;
                                qtotal::Int = 0) where T
    ci    = ChargeInfo{U1Symmetry}(0, :twice_Sz)
    phys = xxz_phys_leg(ci)
    return random_charged_mps(T, phys, N, chi; qtotal = qtotal)
end

```

Local operators for measurements.

```

"""Single-site operator as a (2,2) dense matrix, embedded later
via apply_local_operator_charged or measure_local_observable."""
xxz_local_op(symbol::Symbol; T = Float64) =
    Dict{:Z => T[0.5 0; 0 -0.5],
         :X => T[0 0.5; 0.5 0],
         :Sp => T[0 1; 0 0], :Sm => T[0 0; 1 0],
         :I => T[1 0; 0 1]}[symbol]

```

Note that S^z and S^x are charge-diagonal (preserve q) and charge-changing ($\Delta q = \pm 2$) respectively. $\langle S^z(t) \rangle$ is the only observable the manuscript needs and is fully accessible in sector-conserving form.

8 Module integration and exports

New module file: `U1/src/QuenchDynU1.jl`.

```

module QuenchDynU1

using LinearAlgebra
using Random
using JLD2

include("Symmetry/chargeinfo.jl")
include("Symmetry/legcharge.jl")
include("Symmetry/legpipe.jl")

include("Tensor/chargedtensor.jl")

```

```

include("Tensor/adapters.jl")
include("Tensor/linalg.jl")
include("Tensor/contraction.jl")
include("Tensor/decomposition.jl")

include("MPS/tensorops.jl")
include("MPS/builders.jl")
include("MPS/io.jl")
include("MPS/padding.jl")

include("Algorithms/solvers.jl")
include("Algorithms/algorithms.jl")

include("Models/xxz.jl")

# -- exports (the only XXZ-specific ones) -----
export xxz_phys_leg, xxz_mpo_bond_leg
export build_xxz_mpo_charged
export xxz_neel_state, xxz_random_charged_mps, xxz_local_op

# (plus the generic exports identical to U1.jl's, minus DSF/Models/ED ones)

end # module

```

Project.toml. Copy from KagomeDSF/U1/Project.toml, change name to QuenchDynU1, allocate a fresh UUID. Drop TensorOperations from [deps] since the U(1) code does not actually use it. Keep JLD2, TOML (for any future config loaders).

9 Validation plan

The U(1) layer carries its own test suite at KagomeDSF/U1/test/. We copy it (renaming where it references U1 as a module name), then *add* XXZ-specific tests.

9.1 Inherited generic tests (copy + rename)

File	What it checks
<code>test.chargeinfo.jl</code>	<code>make_valid</code> , $U(1)$ vs $\mathbb{Z}_n$ edge cases.
<code>test.legcharge.jl</code>	Constructors, <code>is_contractible</code> , <code>conj</code> , <code>slice</code> .
<code>test.legpipe.jl</code>	<code>q_map</code> consistency, fusion-of-fusions.
<code>test.chargedtensor.jl</code>	Class invariants, accessors, constructor validation.
<code>test.adapters.jl</code>	<code>to_dense</code> $\circ$ <code>from_dense</code> round-trip; off-sector projection mass.
<code>test.linalg.jl</code>	<code>norm</code> , <code>dot</code> , scalar mul, <code>axpby</code> .
<code>test.contraction.jl</code>	<code>tensordot</code> vs dense, <code>combine/split_legs</code> round-trip.
<code>test.decomposition.jl</code>	<code>svd_charged</code> vs dense SVD, truncation policy.
<code>test.tensorops.jl</code>	Env build / update consistency; orthogonality after <code>make_canonical!</code> .
<code>test.solvers.jl</code>	Lanczos ground state vs dense; Krylov-exp vs dense <code>exp(-iHt)</code> .
<code>test.algorithms.jl</code>	End-to-end DMRG/TDVP on a small generic Heisenberg-like Hamiltonian.
<code>test.builders.jl</code>	<code>product_state_charged</code> , <code>random_charged_mps</code> (after pruning sawtooth bits).
<code>test.io.jl</code>	JLD2 round-trip.

9.2 New XXZ-specific tests

`test.xxz_mpo_charged.jl`.

1. For $N \in \{4, 6, 8\}$, $\Delta \in \{0.5, 1.0, 2.0\}$, random h_j with $|h_j| \leq 0.1$, and uniform h : verify `to_dense(build_xxz_mpo_charged(...)) == build_xxz_mpo(...).tensors[i]` block by block, to tolerance 10^{-12} .
2. Verify `validate_charges` passes on every site.
3. Verify the discarded mass returned by `from_dense` is $< 10^{-13}$.

`test.xxz_dmrg_charged.jl`.

1. For $N = 8$, several Δ : run `dmrg_sweep!` starting from `xxz_neel_state`. Compare ground-state energy against:
 - dense DMRG (`QuenchDyn.dmrg_sweep`), and
 - ED (`build_xxz_hamiltonian + diagonalize`),
to 10^{-8} at $\chi = 32$.
2. Verify total charge is conserved exactly across all sweeps (`qtotal == 0` on the active site).

`test.xxz_tdvp_charged.jl`.

1. Trotter quench on $N = 8$: prepare ground state of $H_i(\Delta_i = 0.5, h)$, evolve under $H_f(\Delta_f = 2.0)$ for $t \in [0, 5]$ at $dt = 0.05$. Compare $\langle S_i^z(t) \rangle$ profiles vs ED (`ed.evolve_and_measure`), to 10^{-6} at $\chi = 64$.

2. Compare against dense TDVP (`QuenchDyn.tdvp_sweep`) at same parameters; verify agreement to 10^{-8} .
3. Check norm preservation in 1-site TDVP (should be machine precision).
4. Check energy drift in 2-site TDVP (should match dense baseline).

`test_quench_protocol_charged.jl`.

1. Reproduce the manuscript’s Fig. 1/2 setup on $N = 12$ (still ED-comparable): DMRG with h_j disorder, quench to clean H_f , evolve, plot $\langle S_i^z(t) \rangle$ and compare ED-style profile. This is a smoke test for the full protocol, not a unit test.

9.3 Drift benchmarks

Mirror the existing `tests/benchmark_tdvp_drift.jl` for the U(1) path. We expect (i) lower memory at the same χ and (ii) the same qualitative drift behaviour — the U(1) layer does not change the truncation policy, so 2-site TDVP still loses norm via SVD and 1-site TDVP still preserves it.

10 Open questions and risks

10.1 MPO convention drift

The XXZ MPO in `Quench_dyn` uses the upper-triangular FSM convention; the sawtooth MPO in `KagomeDSF/U1` also uses an upper-triangular form, but the dense reference in `KagomeDSF`’s non-U(1) builder is lower-triangular. This was a real bug source there (`docs/mpo_convention_fix.md` discusses it). Our path is safer: we only invoke the U(1) projection on tensors produced by our own `build_xxz_mpo`, whose convention is documented and consistent. **Action:** during Phase E, the very first thing to check is that `from_dense` returns the expected non-zero qindex tuples on a single bulk site (`nblocks_stored == 9` for the XXZ W : 1 identity-diagonal entry on each of the three $q=0$ bond pairs, plus $\{(2, 5), (3, 5), (4, 5)\}$ on the right column, plus $\{(1, 2), (1, 3), (1, 4)\}$ in the first row).

10.2 Padding compatibility for 1-site TDVP

The dense 1-site routines in `Quench_dyn` cannot grow χ and require padding from a 2-site warm start. In the U(1) case, padding must preserve charge sectors — the existing `pad_mps_charged` in `KagomeDSF/U1/src/MPS/padding.jl` already does this. **Action:** test `xxz_neel_state` $\rightarrow$ `pad_mps_charged` $\rightarrow$ 1-site DMRG end-to-end as part of Phase F.

10.3 Numerical type uniformity

The U(1) layer parameterises tensors over the element type `T`. For real-time TDVP we will need `T = ComplexF64`. DMRG can use `Float64`. The transition from a real DMRG ground state to complex TDVP evolution happens by widening the type when we build the post-quench state — the dense code handles this implicitly; in the U(1) port we may need an explicit `convert_eltype` helper. **Action:** verify on a small case during Phase D–E.

10.4 Performance expectations

For 1D spin- $\frac{1}{2}$ XXZ in the $m = 0$ sector, the typical U(1) speedup factor over dense MPS is $3\text{--}5\times$ in both memory and per-sweep time at fixed χ . At fixed cost, that translates to a factor $\sim \sqrt{3}$ to $\sqrt{5}$ in effective χ — enough to reach noticeably larger N in the ordered regime, but not orders of magnitude. **Reality check:** if the symmetry-broken regime in the manuscript demands $\chi \sim 10^3\text{--}10^4$, the U(1) port alone may not be sufficient; we would then also need block-aware threading (`Parallel`/sub-package, currently out of scope) or a different algorithm (controlled-bond-expansion TDVP, basis transformation). This is a decision for after we have measured concrete performance.

10.5 Bunching the $q = 0$ bond states

The five XXZ MPO bond states include three with $q = 0$ (Id-start, Sz-pending, Id-finish). They could be bunched into a single qindex of dimension 3, halving the number of off-diagonal blocks the contractor walks through. The sawtooth blueprint defers this to “post-MVP”. We follow the same policy: get correctness first with five unbunched qindices, profile, then bunch if the speedup is meaningful.

11 Phased schedule

Phase	Deliverable	Estimate
0	Create <code>Quench_dyn/U1/</code> skeleton, <code>Project.toml</code> , empty <code>QuenchDynU1.jl</code> .	0.5 d
A	Copy <code>Symmetry/</code> (3 files). Run <code>test_chargeinfo.jl</code> , <code>test_legcharge.jl</code> , <code>test_legpipe.jl</code> .	0.5 d
B	Copy <code>Tensor/</code> (5 files). Run <code>test_chargedtensor.jl</code> , <code>test_adapters.jl</code> , <code>test_linalg.jl</code> , <code>test_contraction.jl</code> , <code>test_decomposition.jl</code> .	1 d
C	Copy <code>MPS/</code> (4 files), prune sawtooth helpers from <code>builders.jl</code> . Run <code>test_tensorops.jl</code> , <code>test_builders.jl</code> , <code>test_io.jl</code> .	1 d
D	Copy <code>Algorithms/</code> (2 files). Run <code>test_solvers.jl</code> , <code>test_algorithms.jl</code> on a generic Heisenberg test.	1 d
E	Write <code>Models/xxz.jl</code> , <code>build_xxz_mpo_charged</code> , leg helpers. Add <code>test_xxz_mpo_charged.jl</code> .	1–1.5 d
F	Add <code>xxz_neel_state</code> , <code>xxz_random_charged_mps</code> , local-op helper. Add <code>test_xxz_dmrg_charged.jl</code> . Confirm DMRG ground state vs dense and ED on $N = 8$.	1 d
G	Add <code>test_xxz_tdvdp_charged.jl</code> : 2-site TDVP quench vs dense and ED on $N = 8$ and $N = 12$.	1–1.5 d
H	Padding + 1-site TDVP path: <code>pad_mps_charged</code> round-trip, then 1-site TDVP smoke test.	0.5 d
I	End-to-end protocol script reproducing manuscript Fig. 1/2 setup on $N = 16$ (out of ED reach) at moderate χ . Visual comparison of $\langle S_i^z(t) \rangle$ vs dense TDVP.	1 d
J	Drift benchmark: <code>benchmark_tdvdp_drift_charged.jl</code> mirroring the dense one.	0.5 d
Total		$\sim 9\text{--}11$ working days

The phasing is intentionally bottom-up: every layer is tested independently before the next is added. Phases A–D consist almost entirely of copy and test execution; Phase E is the only design work (the XXZ MPO charged builder, ~ 100 LOC of actual code plus ~ 200 LOC of tests and docstrings). Phases F–J are integration and validation.

12 What this plan does *not* commit to

- Replacing the dense code under `src/`. The dense path stays as the small- N reference and is what the existing tests and scripts consume. Both paths coexist.
- Enforcing the residual $\mathbb{Z}_2$ parity in the $m=0$ sector. As argued in the manuscript, parity is what we want to *resolve*; baking it into the tensor structure would defeat the purpose.
- Multi-threaded execution (`Parallel/` sub-package in the sawtooth code). Deferred until the serial $U(1)$ path proves insufficient.

- Bunching the $q = 0$ MPO bond states. Deferred until profiling warrants it.
- Spectral-function / DSF infrastructure. Not needed for the quench manuscript; the DSF/ sub-package is left in `KagomeDSF/U1/`.

13 Summary

The $U(1)$ port consists of ~ 5400 lines of generic block-sparse machinery copied verbatim from `KagomeDSF/U1/`, plus ~ 300 lines of native XXZ-specific code (`build_xxz_mpo_charged` and helpers). The MPO charge structure is determined by the existing $\chi_{\text{MPO}} = 5$ FSM with bond charges $(0, +2, -2, 0, 0)$, and the construction reduces mechanically to “build dense W , project with `from_dense`”. No new algorithms are needed; `dmrg_sweep!`, `tdvp_sweep!`, and `tdvp_sweep_one_site!` carry over with identical sweep mechanics. Validation pairs every new path against both the existing dense baseline and ED on small N . Estimated effort: ~ 10 working days from skeleton to a working end-to-end quench protocol script at $N = 16$ outside ED reach.